Bölüm 4: İş Parçacıkları (Threads) ve Çok Çekirdekli Programlama

1. İş Parçacığı (Thread) Nedir? Genel Bakış ve Analoji

Günümüzdeki modern bilgisayarlarda ve mobil cihazlarda çalışan uygulamaların çoğu **çoklu iş parçacıklı (multithread)** bir yapıya sahiptir.

Thread (İş Parçacığı): Merkezi işlem biriminin (CPU) kullanımındaki en temel ve en küçük yürütme birimidir.

- Her bir Thread kendine ait; **Thread ID, Program Counter (PC), bir grup Register (Yazmaç) ve Stack (Yığın)** yapısına sahiptir.
- **Paylaşılanlar:** Aynı process (işlem) içindeki thread'ler process'in **kodunu (code), veri alanını (data) ve diğer sistem kaynaklarını (ör: açık dosyalar)** ortaklaşa kullanırlar.

> 💡 ***Analoji (Restoran Mutfağı):*** *Bir sürecin (Process) koca bir "Restoran Mutfağı" olduğunu düşünelim. Mutfaktaki tezgahlar, dolaplar ve malzemeler process'in hafızasıdır (Bellek / Data). "Thread"ler ise bu mutfakta çalışan* ***Aşçılardır****. Her aşçı aynı anda farklı bir yemek yapabilir (Biri çorba karıştırır, diğeri sebze doğrar). Mutfak kaynaklarını kendi aralarında paylaşarak işi çok daha hızlı ve paralel (veya eşzamanlı) bitirirler.*

---

2. Multithread (Çoklu İş Parçacığı) Programlamanın 4 Avantajı

Uygulamaların tek process içinde birden çok iş parçacığıyla tasarlanmasının ana sebepleri:

1. **Responsiveness (Cevap Verebilirlik):** Bir thread bloke olsa veya uzun bir hesaplama yapsa bile, uygulamanın diğer kısmı çalışmaya devan eder. (Örn: Word'de metin girilirken arkaplanda imla denetimi yapan thread).
2. **Resource Sharing (Kaynak Paylaşımı):** Process'ler kendi aralarında iletişim için paylaşımlı belleğe (shared memory) veya mesajlaşmaya ihtiyaç duyarken; aynı process içindeki Thread'ler varsayılan olarak **belleği ve kaynakları otomatik paylaşırlar**.
3. **Economy (Ekonomi):** Yeni bir tam process oluşturmak, bellek ve kaynak tahsisi gerektirdiği için çok maliyetli iken; aynı kodu/veriyi paylaşan bir **Thread oluşturmak ve Thread arası bağlam geçişi (Context Switch) yapmak çok daha hızlı ve maliyetsizdir**.
4. **Scalability (Ölçeklenebilirlik):** Çok işlemcili (Multiprocessor / Multicore) donanımlarda farklı thread'ler farklı çekirdeklerde **paralel** olarak çalışarak işlem gücünden tam faydalanır.

> ❓ ***Hoca Sorabilir (Kısa Cevap / Avantajlar):*** *"Process oluşturmak yerine neden Thread oluşturmak daha ekonomiktir?"* ✅ ***Cevap:*** *Çünkü thread'ler ait oldukları process'in bellek adres alanını ve kaynaklarını ortak kullanırlar. Yeniden bellek/alan tahsisi gerekmediği için Thread oluşturmak ve iki Thread arasında bağlam geçişi (Context Switch) yapmak, tam bir Process oluşturmaktan ve process'ler arası geçişten kat kat daha az maliyetli/hızlıdır.*

---

3. Çok Çekirdekli (Multicore) Programlama ve Temel Zorluklar

Tek bir elektronik çip içerisine birden fazla çekirdek (core) yerleştirilmiş sistemlere multithreaded yaklaşımlar büyük güç katar. Ancak bu gücü kullanabilmek için programcıların aşması gereken **5 temel zorluk** vardır:

1. **Identifying Tasks (Görevlerin Belirlenmesi):** Hangi alt görevlerin birbirinden bağımsız eşzamanlı çalışacağının tespit edilmesi.
2. **Balance (Dengeleme):** Belirlenen görevlerin aynı iş yükünde ve eşit oranda dağıtılması.

3. **Data Splitting (Veri Bölme):** Farklı çekirdeklerde işlenecek verinin görevlere uygun şekilde parçalanması.
4. **Data Dependency (Veri Bağımlılığı):** Görevler arası veri bağımlılıklarında, bir görev diğerinden senkronizasyon bekliyorsa bu sürecin (synchronous) dikkatlice ayarlanması.
5. **Testing and Debugging (Test ve Hata Ayıklama):** Birçok farklı yürütme yolunun ihtimali olduğu için, sıralı koda kıyasla hatanın (bug) bulunup düzeltilmesinin çok zor olması.

Eşzamanlılık (Concurrency) vs Paralellik (Parallelism) Karşılaştırması

> ❓ ***Hoca Sorabilir (Karşılaştırma):*** *"Eşzamanlı (Concurrent) yürütme ile Paralel (Parallel) yürütmenin farkı nedir?"* ✅ ***Cevap:*** *Paralellikte birden fazla görev* ***aynı zaman diliminde (aynı anda) farklı donanım çekirdeklerinde*** *fiziksel olarak gerçekleşir. Eşzamanlılıkta ise görevlerin* ***aynı anda bitirilmesi için birimleri paylaşılarak ilerletilmesidir*** *(Tek çekirdekte görevler arası çok hızlı geçiş - context switch yapılarak paralelmiş hissi verilmesi de bir concurrency örneğidir). Paralellik için çok çekirdek gerekir ancak Eşzamanlılık için gerekmez.*

Özellik	Eşzamanlılık (Concurrency)	Paralellik (Parallelism)
Temel Tanımı	Çoklu işlemlerin adaletli/kısa aralıklarla kaynak paylaşıp *aynı anda ilerletilmesi.*	Çoklu işlemlerin fiziksel olarak *aynı anın milisaniyesinde, farklı kaynaklarda* icra edilmesi.
Donanım Şartı	Tek Çekirdek (Single Core) dahi olsa yapılabilir (Hızlı CPU scheduling ile yanıltsama).	Kesinlikle Çok Çekirdek (Multi-core) veya Çok İşlemcili sisteme ihtiyaç vardır.

(Not: Amdahl Kuralı (S), çekirdek sayısı arttıkça sistemdeki hızlanmanın sonsuz olamayacağını, programın mutlaka seri/sırayla çalışması gereken kısmından dolayı (1/S) bir sınıra dayanacağını belirtir.)

---

4. Paralel Çalışma Türleri (Data vs Task)

Bir uygulama paralel yürütme yaparken hibrit çalışabilir, ancak iki ana stratejiye ayrılır:

Paralellik Türü	Çalışma Mantığı	Örnek Senaryo
Data Parallelism (Veri Paralelliği)	**Aynı verinin alt kümeleri** farklı çekirdeklere dağıtılır ve tüm çekirdeklerde **aynı işlem/fonksiyon** uygulanır.	1.000.000 elemanlı dizinin bir yarısını Çekirdek 1'de, diğer yarısını Çekirdek 2'de toplamak. İşlem aynı (toplama), veri farklı alt kümelerde.
Task Parallelism (Görev Paralelliği)	Çekirdeklere veri değil, **farklı görevler (iş parçacıkları)** dağıtılır. Çekirdeklerin icra ettiği kod veya işlem tamamen farklıdır.	Bir Thread istatistik hesaplarken (Çekirdek 1), diğer Thread kullanıcı arayüzünü günceller (Çekirdek 2).

---

5. Multithreading Modelleri (Vurucu Sınav Konusu)

Kullanıcı seviyesindeki Thread'ler (User Threads), işletim sisteminin kernel'inin farkında olduğu Kernel Thread'lerle eşleşmelidir. Bu ilişki kurmanın 3 temel yolu vardır:

Model Adı	Eşleşme Biçimi	Bloklanma Durumu (Kernel Engeli)	Dezavantajı / Yaygınlığı

Many-to-One	*Çok sayıda Kullanıcı ➔ 1 Adet Kernel Thread*	İçlerinden birisi bloke eden sistem çağrısı yaparsa, **TÜM süreç bloke olur** (Thread bekler).	Eşzamanlı donanımları tam kullanamaz. Günümüzde pek tercih edilmez (Eski Solaris vb.).
One-to-One	*1 Adet Kullanıcı ➔ 1 Adet Kernel Thread*	Bir thread bloke olsa da **diğerleri çalışmaya devam eder.** Gerçek eşzamanlılığa izin verir.	Açılan her User Thread için Kernel Thread açmak sistemi yorar. Sistem genel performansını düşürebilir ama **Günümüzde en yaygın (Linux, Windows)** modeldir.
Many-to-Many	*Çok Çok Kullanıcı ➔ Sınırlı/Eşit Kernel Thread*	Kernel başka bir thread'e öncelik vererek sistemin durmasını önlenecek çizelgeleme (scheduling) yapar.	Yönetimi (gerçekleştirimi) çok karmaşık olduğu için zor bir modeldir. Geri planda esnektir.

> ❓ ***Hoca Sorabilir (Boşluk Doldurma / Klasik):*** *"Eğer modern bir işletim sisteminde (ör: Linux veya Windows) her kullanıcı iş parçacığı sadece bir çekirdek iş parçacığıyla eşleniyorsa, bu mimariye ..... modeli denir."* ✅ ***Cevap: One-to-One (Bire Bir)***

---

6. Dolaylı Thread Oluşturma (Implicit Threading) ve Thread Havuzları (Thread Pools)

Multicore işlemcilerin ilerlemesiyle binlerce thread'in manuel yaratılması (yönetimi ve test edilmesi) imkansız hale geldi. Çözüm, thread yönetimini programcıdan alıp **Derleyici (Compiler) ve Run-Time Kütüphanelere** vermektir (*Implicit Threading*). En bilinen en güçlü yöntem ise **Thread Pools (Havuzları)**.

Thread Pools Neden Kullanılır ve 3 Büyük Avantajı Nedir?

Bir sunucuya binlerce istek geldiğinde her istek için sıfırdan "New Thread()" oluşturmak, CPU zamanını ve RAM'i tüketip sistemi çökertebilir. Bu sebeple işlemci, sistem açıldığında belirli sayıda hazır Thread üretip bir "hvuza" koyar. Gelen istekler yeni oluşturulmak yerine havuzdaki **boştaki** thread'e atanır, işi biten thread yeniden havuza döner.

1. Mevcut (halihazırda yaratılmış) bir thread'i kullanarak hizmet vermek, yeni bir thread oluşturma bekleme süresinden **çok daha hızlıdır**.
2. Max eklenecek devasa sayıda eşzamanlı Thread'i sınırlandırmayı sağlayarak (havuz kapasitesi) **sistemin tükenmesini ve çökmesini engeller.**
3. Zamanlanmış (Scheduled) farklı görev stratejileri için altyapı sunar.

> ❓ ***Hoca Sorabilir:*** *"Aynı process içinde sürekli thread oluşturmanın maliyeti process'e göre düşük olsa da, çok sayıda kullanıcının eriştiği bir sunucuda her işlem için anlık yeni bir Thread açmak hangi riskleri taşır? En iyi çözüm stratejisi nedir?"* ✅ ***Cevap:*** *Gelen her yeni isteğin, sınırsız yeni "Thread" açması cihazın RAM'ini veya CPU zamanını tüketip, Response süresini düşürebilir ve "Memory Exhaustion" sonucu sistem çökebilir. En güvenli çözüm* ***Thread Pool (İş Parçacığı Havuzu)*** *kullanmaktır; böylece kaynak yönetimi optimize edilerek aynı anda çalışacak görev sayısı önceden sınırlandırılmış ve sistem güvenceye alınmış olur.*

---

7. Kilit (Anahtar) Kavramlar

- **Pthreads (POSIX):** Kernel veya User seviyesi olabilen, UNIX ve Linux'ların yaygın ve standartlaştırılmış (IEEE) arayüz kütüphanesidir. (Windows hariç, Windows kendi Win32 kütüphanesini kullanır.)

- **Java Threads:** User-level olarak başlar fakat JVM (Android, Linux vs) mimarisinde çalışır. Thread üretmek için `Runnable` arayüzü sıklıkla kullanılır, veya `java.util.concurrent` paketi ile sonuç dönebilen `Callable` çağrılır.
- **Amdahl Kuralı:** Seri çalışan oran (S) büyüdükçe, işlemci eklemek performansı bir yere kadar (1/S) arttırabilir, ondan sonrasında çekirdek katmak teorik hesaplamada bir artış yapmayacaktır.
- **Synchronous vs Asynchronous Threading:**
  - **Asynchronous:** Parent, child thread'i oluşturduktan sonra kendi işine paralelde duraksamadan devem eder. (Veri paylaşımının / bağlılığının AZ olduğu durumlar). Web Sunucuları.
  - **Synchronous (Fork-Join):** Parent, bir child oluşturur ve çalışmasını durdurur. Çocukların hepsinin işi bittiğinde sonuçları alarak çalışmasına döner. Veri bağımlılığının çok yüksek olduğu durumlardır.

---

8. Sınav Öncesi Son Kontrol (Özet)

- [ ] Processler arası geçiş (Context Switch) her zaman, Thread'ler arası bağlam geçişinden **daha maliyetli ve yavaştır.**
- [ ] "Data Parallelism", veri bloklarının çekirdeklere dağıtılmasıdır. "Task Parallelism", aynı veriye bambaşka görevlerin (örn: gui, math, hesaplama) dağıtılmasıdır.
- [ ] Her bir thread kendine has **Thread ID, Yığın (Stack), PC (Program Counter) ve Register Set** barındırır.
- [ ] Bütün Thread'ler, kendilerini sarmalayan *Process'in* dosya alanı, bellek alanı (Data + Text/Code Segment) gibi **tüm ana kaynaklarını tamamen paylaşırlar.** (Ayrıcalıkları yoktur)
- [ ] Sistemin sınırsız Thread oluşturmaktan çökmemesi ve verimlilik adına "Implicit Threading" başlığı altında en popüler önlem **"Thread Pools (Havuzları)"** mimarisidir.
- [ ] Günümüz masaüstü/donanım sistemlerinde en yaygın eşleştirme modeli **One-to-One (Bire bir)** modelidir; zira Many-to-One'da tek bir I/O kesintisi bile programın tüm parçacıklarını kilitleyerek kilitler.